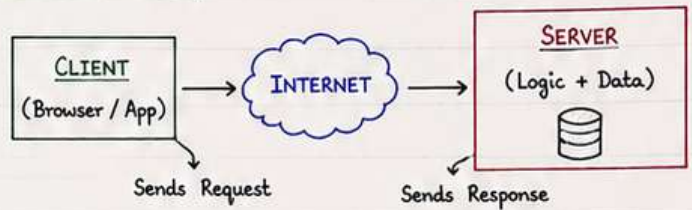


20. TOP 50 BACKEND INTERVIEW CONCEPTS

PAGE 1 : BACKEND FUNDAMENTALS (1 - 7)

1. CLIENT - SERVER ARCHITECTURE

A distributed architecture where clients request services or resources from a server. The server processes the request and sends back a response.

2. REST API

REST (Representational State Transfer) is an architectural style for building APIs using HTTP protocol.

★ KEY POINTS

- Stateless
- Client - Server
- Cacheable
- Layered System
- Uniform Interface
- Code on Demand (optional)

3. HTTP METHODS

- **GET** - Retrieve a resource
- **POST** - Create a new resource
- **PUT** - Update an existing resource (full)
- **PATCH** - Update an existing resource (partial)
- **DELETE** - Delete a resource

METHOD	IDEAL USE
GET	Read data
POST	Create data
PUT	Replace data completely
PATCH	Modify data partially
DELETE	Remove data

4. HTTP STATUS CODES

- 1xx - Informational
- 2xx - Success
- 3xx - Redirection
- 4xx - Client Error
- 5xx - Server Error

COMMON CODES

- 200 OK - Request successful
- 201 Created - Resource created
- 400 Bad Request - Invalid request
- 401 Unauthorized - Authentication required
- 403 Forbidden - Access denied
- 404 Not Found - Resource not found
- 500 Internal Server Error - Server issue

5. STATELESS vs STATEFULSTATELESS

- Server does not store client context.
- Each request contains all necessary data.
- Example: JWT, Token based auth
- Scalable, reliable

STATEFUL

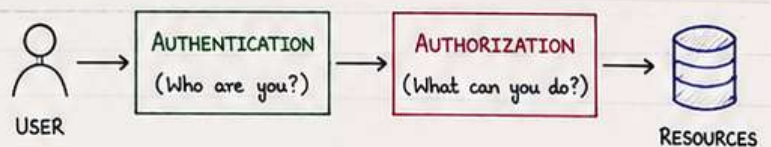
- Server stores client session/context.
- Example: Session ID, Cart in server
- Harder to scale
- Used in traditional apps

★ TIP

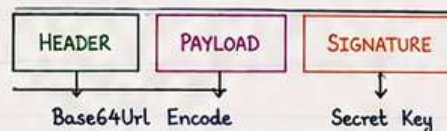
Most REST APIs are Stateless.

6. AUTHENTICATION vs AUTHORIZATION

- **Authentication** - Verifies who you are. (Login / Identity)
- **Authorization** - Verifies what you can do. (Permissions / Roles)

7. JWT (JSON WEB TOKEN)

A compact, URL-safe token used for securely transmitting information between parties.

USED FOR

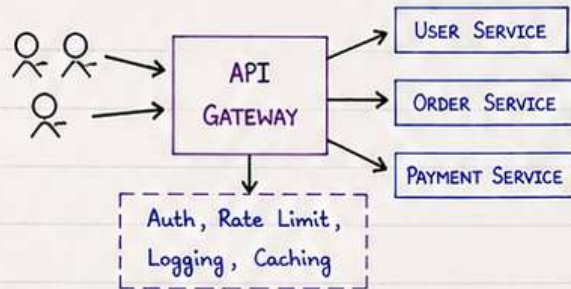
- Authentication
- Information exchange
- Stateless sessions

20. TOP 50 BACKEND INTERVIEW CONCEPTS

PAGE 2 : API & BACKEND DESIGN (8-14)

8. API GATEWAY

- Single entry point for all client requests.
- Routes requests to appropriate services.
- Handles auth, rate limiting, logging, etc.
- Hides internal architecture from clients.

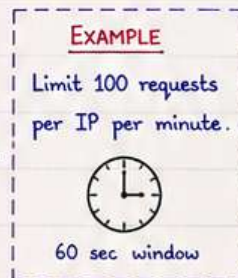


★ TIP

Use API Gateway when you have multiple services or microservices.

9. RATE LIMITING

- Limits number of requests from a client in a given time window.
- Prevents abuse and protects system.
- Implemented in API Gateway or at service level.



10. IDEMPOTENCY

- Multiple identical requests have the same effect as a single one.
- Important for POST/PUT requests.
- Use Idempotency-Key header to ensure uniqueness.

USE CASE

Payment requests, Order creation, Retry mechanisms.

11. PAGINATION

- Break large result sets into smaller pages.
- Improves performance and user experience.
- Common types:
 - Page Number Based
 - Cursor Based
 - Offset Based

EXAMPLE (PAGE NUMBER BASED)

```
GET /api/users?page=2&limit=10
Response:
{
  "data": [...10 users...],
  "page": 2,
  "limit": 10,
  "totalPages": 25,
  "totalItems": 248
}
```

12. API VERSIONING

- Manage changes without breaking existing clients.
- Versions in URL, Header or Query Param.
- Example:
 - /api/v1/users
 - /api/v2/users

BEST PRACTICE

Add new versions, don't change existing ones.

13. WEBHOOKS

- Server-to-server callbacks.
- Notify other systems about events in real-time.
- Common in payments, CI/CD, CRM, etc.

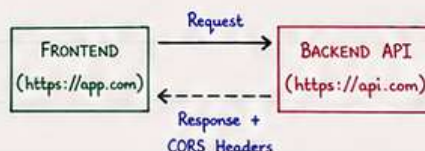


EXAMPLE

Payment gateway sends webhook after payment success or failure.

14. CORS (CROSS ORIGIN RESOURCE SHARING)

- Mechanism that allows restricted resources on a web page to be requested from another domain.
- Controlled by server using HTTP headers.
- Helps in secure cross-origin requests.



COMMON HEADERS

- Access-Control-Allow-Origin
- Access-Control-Allow-Methods
- Access-Control-Allow-Headers

💡 QUICK RECAP (8-14)

- | | |
|--|--|
| ✓ API Gateway → Single entry point to route & secure requests. | ✓ API Versioning → Manage changes safely. |
| ✓ Rate Limiting → Protects from abuse & ensures fair usage. | ✓ Webhooks → Real-time server to server notifications. |
| ✓ Idempotency → Same request multiple times, same result. | ✓ CORS → Secure cross-origin communication. |
| ✓ Pagination → Break large data into smaller chunks. | |

★ Design clean APIs. Think Security, Scalability & User Experience. ☺ ★

20. TOP 50 BACKEND INTERVIEW CONCEPTS

PAGE 3 : DATABASE CONCEPTS (15-21)

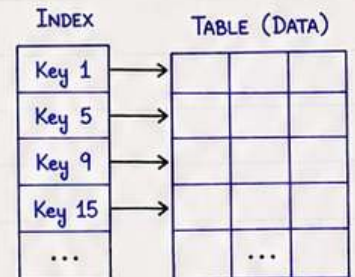
15. SQL vs NoSQL

SQL (RELATIONAL)	NoSQL (NON-RELATIONAL)
- Table based (Rows, Columns) - Fixed schema - ACID compliant - Joins supported - Vertically Scalable - Examples : MySQL, PostgreSQL	- Key-Value / Document / Column / Graph - Flexible schema - BASE (Eventually consistent) - Joins not common - Horizontally Scalable - Examples : MongoDB, Cassandra, Redis

16. INDEXING

- Improves query performance by reducing search time.
- Index is a data structure that helps DB find rows faster.
- Types :
 - Primary Index (On PK)
 - Unique Index
 - Composite Index
 - Full Text Index

How Index Works ?



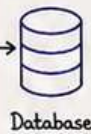
★ TIP

Indexing speeds up READS but may slow down WRITES.

17. TRANSACTIONS & ACID

A transaction is a sequence of one or more SQL operations treated as a single unit of work.

- A** Atomicity - All or nothing.
C Consistency - DB remains in valid state.
I Isolation - Concurrent txns don't interfere.
D Durability - Once committed, changes persist even after failure.



Database

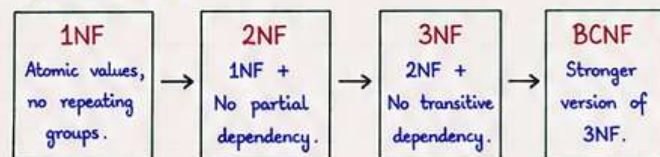
18. ISOLATION LEVELS

- Isolation levels control how transactions behave concurrently.

LEVEL	DESCRIPTION	DIRTY	NON-REPEATABLE	PHANTOM
READ UNCOMMITTED	Highest concurrency, lowest isolation	✓	✓	✓
READ COMMITTED	Prevents dirty reads	✗	✓	✓
REPEATABLE READ	Prevents dirty reads & non-repeatable reads	✗	✗	✓
SERIALIZABLE	Highest isolation (Transactions run serially)	✗	✗	✗

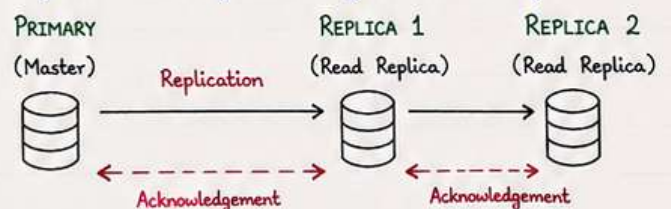
19. DATABASE NORMALIZATION

- Process of organizing data to reduce redundancy.
- Improves data integrity.
- Normal Forms :



20. REPLICATION

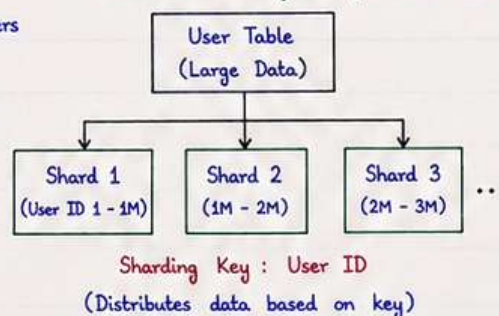
- Copying data from one DB server to another.
- Improves availability, reliability & read scaling.



21. SHARDING

- Splitting data across multiple DB servers (Shards).
- Improves write & read scalability.
- Types :
 - Horizontal Sharding (By rows)
 - Vertical Sharding (By columns)

Horizontal Sharding Example (User Table)



★ REMEMBER

- Use SQL for complex relationships.
- Use NoSQL for flexible & high scale needs.
- Good Schema + Index = High Performance.



Strong Database Knowledge is the foundation of a great backend engineer!

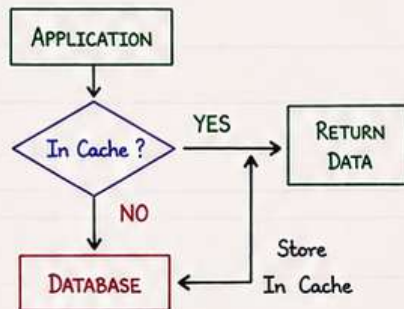


20. TOP 50 BACKEND INTERVIEW CONCEPTS

PAGE 4 : CACHING & PERFORMANCE (22-28)

22. CACHE

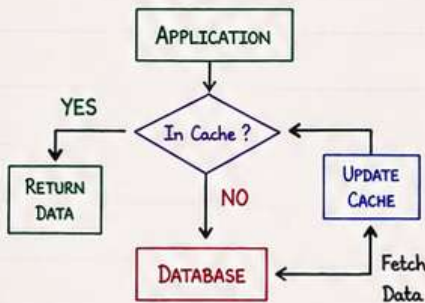
- Cache stores frequently used data in memory for fast access.
- Reduces load on DB and improves response time.
- Types: In-Memory (Redis, Memcached), Browser Cache, CDN Cache.



★ TIP

- Cache improves READ performance.
- Always set proper TTL (Time To Live).
- Don't cache everything!

23. CACHE-ASIDE PATTERN (LAZY LOADING)



STEPS

1. App looks for data in cache.
2. If found → return.
3. If not found → fetch from DB.
4. Store in cache.
5. Return data to client.

24. WRITE-THROUGH vs WRITE-BACK

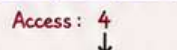
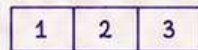
WRITE-THROUGH	WRITE-BACK (WRITE-BEHIND)
• Data is written to cache and DB at the same time. • More consistent. • Slower writes. • Use case: Banking, Orders.	• Data is written to cache first, then to DB later. • Faster writes. • Risk of data loss. • Use case: Logs, Analytics.

25. CACHE EVICTION POLICIES

- LRU - Least Recently Used
- LFU - Least Frequently Used
- FIFO - First In First Out
- TTL - Time To Live (Expiry based)
- RANDOM - Randomly remove items

LRU EXAMPLE

Capacity = 3



1 is evicted (LRU)

26. REDIS

- In-memory data structure store.
- Supports Strings, Hashes, Lists, Sets, Sorted Sets, etc.
- Used for caching, sessions, rate limiting, leaderboards.
- Features: Persistence, Pub/Sub, Replication, TTL.

COMMON REDIS COMMANDS

```

SET key value
GET key
DEL key
EXPIRE key 60
HSET user:1 name abhi
HGET user:1 name
LPUSH list 1
LRANGE list 0 -1
  
```

27. CDN (CONTENT DELIVERY NETWORK)

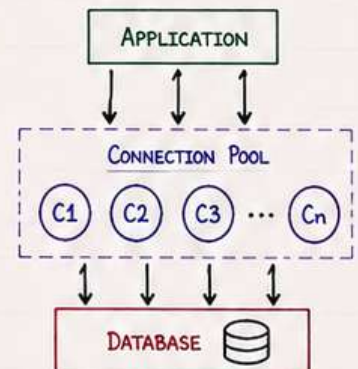
- Distributes content across multiple edge servers.
- Delivers content closer to users.
- Improves load time & availability.
- Reduces load on origin server.



If content in CDN → Serve.
If not → fetch from origin.

28. CONNECTION POOLING

- Reuses existing DB connections.
- Avoids overhead of creating new connections.
- Improves performance & handles high concurrency.
- Configurable pool size.



★ QUICK RECAP (22-28)

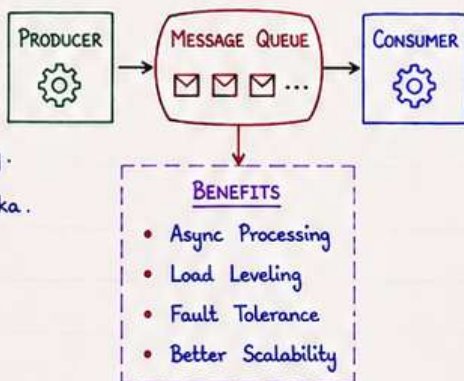
- | | |
|--|--|
| <ul style="list-style-type: none"> ✓ Cache → Store data for fast access. ✓ Cache-Aside Pattern → Lazy loading to cache. ✓ Write-Through → Write to cache + DB together. ✓ Write-Back → Write to cache first, DB later. | <ul style="list-style-type: none"> ✓ Eviction Policies → LRU, LFU, FIFO, TTL, Random. ✓ Redis → Powerful in-memory data store. ✓ CDN → Deliver content closer to users. ✓ Connection Pooling → Reuse DB connections. |
|--|--|

20. TOP 50 BACKEND INTERVIEW CONCEPTS

PAGE 5 : ASYNC & DISTRIBUTED SYSTEMS (29-35)

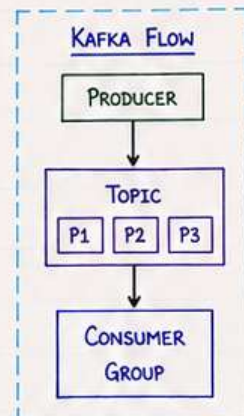
29. MESSAGE QUEUE

- Enables systems to communicate asynchronously.
- Decouples producer and consumer.
- Improves reliability and scalability.
- Examples: RabbitMQ, SQS, Kafka.



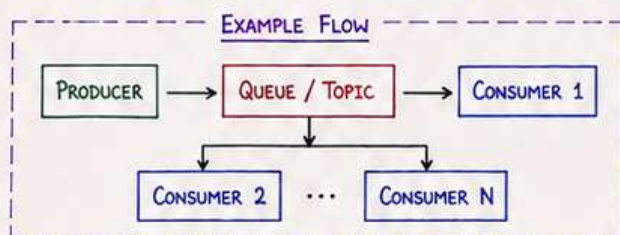
30. KAFKA

- Distributed event streaming platform.
- High throughput, scalable, fault tolerant.
- Stores events in topics.
- Key components:
 - Producer
 - Broker
 - Topic
 - Consumer



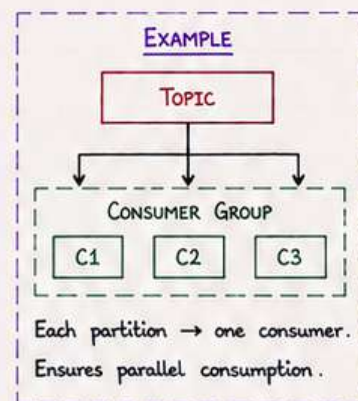
31. PRODUCER / CONSUMER

- **Producer**: Sends messages to the queue/topic.
- **Consumer**: Reads and processes messages.
- Multiple consumers can read for parallel processing.
- Ensures high availability and scalability.



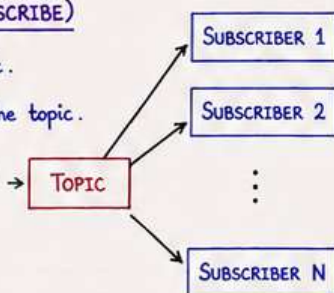
32. CONSUMER GROUPS

- Consumers are grouped to share the load.
- Each message is consumed by only one consumer in the group.
- Enables horizontal scaling of consumers.



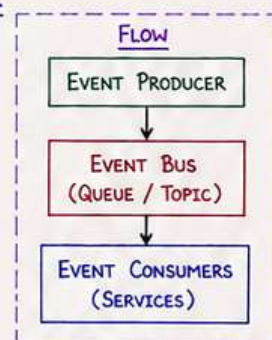
33. PUB / SUB (PUBLISH / SUBSCRIBE)

- Publisher sends messages to a topic.
- Subscribers receive messages from the topic.
- One-to-Many communication model.
- Used in real-time notifications, chat apps, live updates, etc.



34. EVENT-DRIVEN ARCHITECTURE

- System reacts to events.
- Components communicate via events.
- Loosely coupled, scalable and resilient.
- Common in microservices, IoT, real-time systems.



35. SYNCHRONOUS vs ASYNCHRONOUS

SYNCHRONOUS	ASYNCHRONOUS
Request → Wait → Response	Request → Continue → Later Response
Blocking call	Non-blocking call
Lower throughput	Higher throughput
Tight coupling	Loose coupling
Ex: REST API call	Ex: Message Queue, Kafka

★ INTERVIEW TIP

- Understand the trade-offs between consistency and availability.
- Know when to use Queue vs Kafka.
- Event-driven systems are key for building scalable architectures.
- Focus on fault tolerance and idempotency in async systems.



Think Async. Build Scalable. Design Resilient Systems.

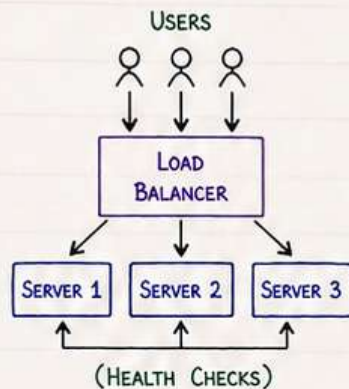


20. TOP 50 BACKEND INTERVIEW CONCEPTS

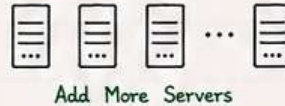
PAGE 6 : SCALABILITY & RELIABILITY (36-43)

36. LOAD BALANCER

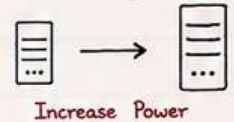
- Distributes incoming traffic across multiple servers.
- Improves availability and prevents any single server from being overwhelmed.
- Types : L4 (Transport) L7 (Application)

37. HORIZONTAL vs VERTICAL SCALINGHORIZONTAL SCALING

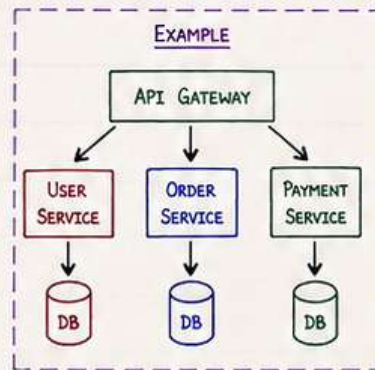
- Add more machines (scale out).
- Handles more traffic.
- Cost effective.

VERTICAL SCALING

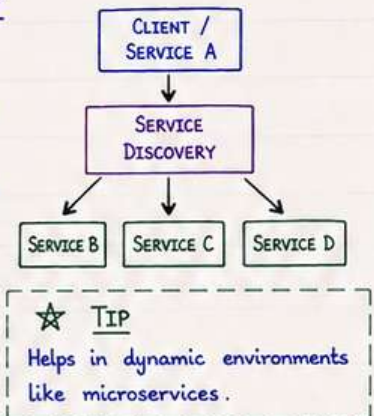
- Add more power to existing machine (scale up).
- Limited by hardware.

38. MICROSERVICES

- Application is broken down into small, independent services.
- Each service has its own DB.
- Improves scalability, flexibility, and team productivity.
- Communication via APIs.

39. SERVICE DISCOVERY

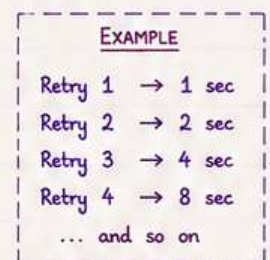
- Mechanism to find and communicate with services automatically.
- Removes hard-coded service locations.
- Tools : Eureka, Consul, etcd, Zookeeper.

40. CIRCUIT BREAKER

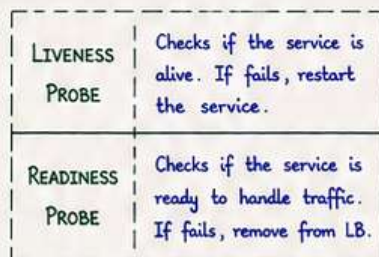
- Prevents cascading failures by stopping calls to failing services.
- States : Closed → Open → Half-Open → Closed
- Tools : Resilience4j, Hystrix.

41. RETRY & EXPONENTIAL BACKOFF

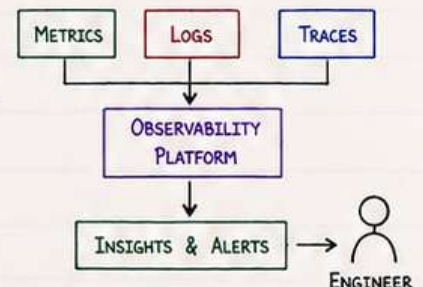
- Retry failed requests to handle transient errors.
- Exponential backoff increases wait time between retries.
- Prevents overwhelming downstream services.

42. HEALTH CHECKS

- Periodic checks to ensure services are running fine.
- Used by Load Balancers and Orchestrators.
- Types : Liveness, Readiness.

43. OBSERVABILITY

- Ability to understand the internal state of systems.
- Pillars :
 - Metrics
 - Logs
 - Traces

★ QUICK RECAP (36-43)

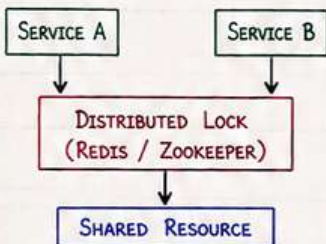
- | | |
|--|---|
| ✓ Load Balancer → Distribute traffic evenly. | ✓ Circuit Breaker → Prevent cascading failures. |
| ✓ Horizontal Scaling → Add more servers. | ✓ Retry & Backoff → Handle transient failures. |
| ✓ Microservices → Small, independent services. | ✓ Health Checks → Keep system healthy. |
| ✓ Service Discovery → Find services dynamically. | ✓ Observability → Metrics, Logs, Traces = Insights. |

20. TOP 50 BACKEND INTERVIEW CONCEPTS

PAGE 7 : ADVANCED CONCEPTS & CHEAT SHEET (44-50)

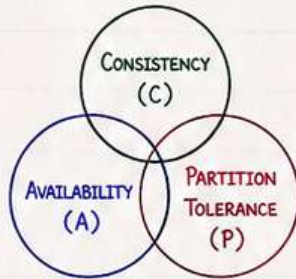
44. DISTRIBUTED LOCK

- Ensures only one process can access a shared resource.
- Prevents race conditions in distributed systems.
- Common using Redis, ZooKeeper, etc.



45. CAP THEOREM

- In a distributed system, you can achieve only two out of three:
- Consistency, Availability, Partition Tolerance.



46. CONSISTENCY MODELS

- Strong Consistency
 - All reads return the latest write.
- Eventual Consistency
 - System will become consistent over time.
- Causal Consistency
 - Guarantees ordering of causally related operations.

★ INTERVIEW TIP

Always mention:

- ☒ Definition
- ☒ Why it is needed
- ☒ How it works
- ☒ Pros / Cons
- ☒ Real-world example



47. DATABASE DEADLOCKS

- Occurs when two or more transactions are waiting for each other indefinitely.
- DB detects and resolves by aborting one transaction.

PREVENTION TIPS

- Keep transactions short.
- Access tables in same order.
- Use proper indexing.
- Avoid long running locks.

48. DISTRIBUTED TRANSACTIONS

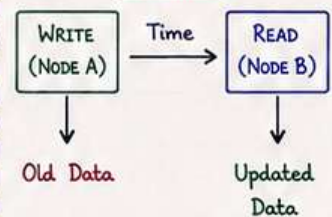
- Transactions that span across multiple services / databases.
- Ensures ACID across systems.
- Common Protocols:
 - 2PC (Two Phase Commit)
 - Saga Pattern
 - TCC (Try-Confirm-Cancel)



49. EVENTUAL CONSISTENCY

- System may not be immediately consistent but will become consistent over time.
- Improves availability and partition tolerance.
- Examples: DynamoDB, Cassandra, Riak.

EXAMPLE



After some time → Both nodes are in sync.

🔥 BACKEND INTERVIEW RAPID REVISION (CHEAT SHEET)

① REST



Stateless, Scalable, Cacheable APIs. Use proper HTTP methods & status codes.

② REDIS (CACHE)



In-memory key-value store. Improves performance. Use Cache-Aside pattern.

③ KAFKA (MQ)



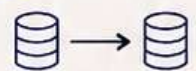
Distributed event streaming. High throughput, decouples services.

④ DATABASE INDEX



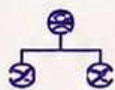
Speeds up read queries. Use proper indexes, avoid over-indexing.

⑤ SHARDING



Horizontal scaling by splitting data across multiple DBs.

⑥ LOAD BALANCER



Distributes traffic across servers. Improves availability.

⑦ CIRCUIT BREAKER



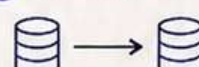
Protects from cascading failures. Improves system stability.

⑧ MESSAGE QUEUE



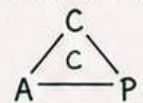
Decouple services. Handles spikes & improves reliability.

⑨ REPLICATION



Copies data to multiple nodes. Improves availability & read scalability.

⑩ CAP THEOREM



You can have only two out of three. Choose what matters most for your system.



Strong Backend Knowledge + System Design Thinking = Scalable, Reliable & Efficient Systems.

